



**PIMPRI CHINCHWAD EDUCATION TRUST'S
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING**

*NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune*

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 3: Fractional Knapsack Problem

Name	: Aryan Ligade
PRN	: 124B1F155
Branch	: Information Technology (IT)
Semester	: V
Batch	: A4
Course Code	: BIT25PC04
Course Name	: Design & Analysis of Algorithms Laboratory
Guided By	: Dr. Rajesh N. Phursule

Assignment No. 3: Fractional Knapsack Problem

1. Problem Statement:

Emergency Relief Supply Distribution: A devastating flood has hit multiple villages in a remote area, and the government, along with NGOs, is organizing an emergency relief operation. A rescue team has a limited-capacity boat that can carry a maximum weight of W kilograms. The boat must transport critical supplies, including food, medicine, and drinking water, from a relief center to the affected villages.

Each type of relief item has:

- A weight (w_i) in kilograms.
- Utility value (v_i) indicating its importance (e.g., medicine has higher value than food).
- Some items can be divided into smaller portions (e.g., food and water), while others must be taken as a whole (e.g., medical kits).

2. Goals:

1. Implement the Fractional Knapsack algorithm to maximize the total utility value of the supplies transported.
2. Prioritize high-value items while considering weight constraints.
3. Allow partial selection of divisible items (e.g., carrying a fraction of food packets).
4. Ensure that the boat carries the most critical supplies given its weight limit W .

3. Course Objectives:

- 3.1. To know the basics of computational complexity of various algorithms.
- 3.2. To select appropriate algorithm design strategies to solve real-world problems.

4. Course Outcomes:

After learning the course, students will be able to:

- 4.1. Analyze the asymptotic performance of algorithms.
- 4.2. Solve computational problems by applying suitable paradigms such as Greedy method.

5. Theory:

The Fractional Knapsack Problem is a classic optimization problem: given a set of items—each with weight w_i and value v_i —and a maximum capacity W , the objective is to maximize total value by selecting items (or fractions of them) up to capacity.

Unlike the 0/1 Knapsack, the fractional version allows partial selection, making it solvable optimally in polynomial time.

Algorithm (Greedy Strategy):

1. Compute the value-to-weight ratio v_i / w_i for each item.
2. Sort items in descending order of this ratio.
3. Fill the knapsack:
 - Take items fully if they fit.
 - If capacity runs out, take the exact fraction needed of the current item (only if divisible).
 - Skip indivisible items that do not fit.

Working of Greedy:

1. **Greedy-choice property:** Choosing the highest ratio item at each step is always part of an optimal solution.
2. **Optimal substructure:** Once part of the knapsack is filled, the remaining capacity forms a smaller instance of the same problem.

Item	Weight (kg)	Utility	Taken	Value
 Medicine	5	50	Full	50
 Food	10	30	Full	
 Water	20	20	Partial	up to 20

 Boat capacity: 17 kg

Figure 5.1: Priority-based Item Selection and Boat Capacity Allocation

6. Flowchart:

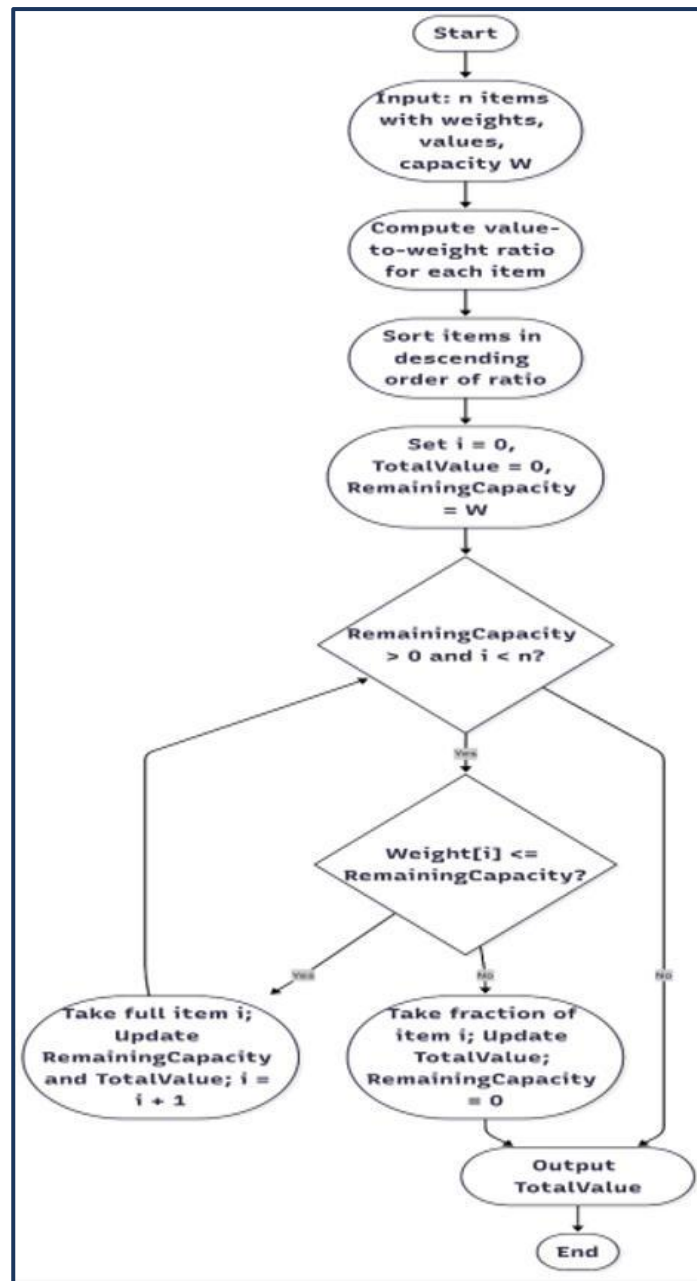


Figure 6.1: Flowchart of Fractional Knapsack Greedy Strategy

7. Time Complexity:

Operation	Time Complexity	Explanation
Ratio computation	$O(n)$	Calculating value-to-weight ratio (v_i / w_i) for all n items
Sorting	$O(n \log n)$	Sorting items in descending order of their ratios
Selection loop	$O(n)$	Iterating through items and placing into knapsack
Total Time Complexity	$O(n \log n)$	Dominated by the sorting phase

8. Pseudo Code for Fractional Knapsack:

```

function MaxUtilFractional(items, W):
    Input: items = list of (weight w, value v, isDivisible), capacity W
    Output: maximum total utility value

    Compute ratio  $r[i] = v[i] / w[i]$  for each item
    Sort items by descending  $r[i]$ 
    totalValue = 0
    remainingCapacity = W

    for each item in sorted items:
        if remainingCapacity == 0:
            break
        if item.weight <= remainingCapacity:
            totalValue += item.value
            remainingCapacity -= item.weight
        else if item.isDivisible:
            fraction = remainingCapacity / item.weight
            totalValue += fraction * item.value
            remainingCapacity = 0
        // else if indivisible and can't fit, skip
    end for

    return totalValue
end function

```

9. Sample Example & Step-by-Step Allocation:

Boat capacity: $W = 17$ kg. Total potential utility if all fit = 100.

Item	Weight (kg)	Utility	Divisible	Taken	Value Obtained
Medicine	5	50	No	Full	50
Food	10	30	Yes	Full	30
Water	20	20	Yes	Partial (if needed)	up to 20

Step 1: Calculate utility per kg (Ratio v_i / w_i):

- **Medicine:** $50 / 5 = 10$ utility/kg
- **Food:** $30 / 10 = 3$ utility/kg
- **Water:** $20 / 20 = 1$ utility/kg

Preference order: Medicine $\rightarrow$ Food $\rightarrow$ Water

Step 2: Fill the boat:

- **Take Medicine fully:** Weight used = 5 kg, Utility = 50, Remaining capacity = $17 - 5 = 12$ kg
- **Take Food fully:** Weight = 10 kg, divisible. Fits completely. Utility = 30, Remaining capacity = $12 - 10 = 2$ kg
- **Take Water partially:** Weight = 20 kg, divisible. Only 2 kg fits. Utility = $20 \times (2 / 20) = 2$. Remaining capacity = 0 kg.

Code and Output:

Repository: <https://github.com/aryanligade24/DAA-Lab-Assignmentss.git>

10. Conclusion:

The Fractional Knapsack Algorithm maximizes utility by prioritizing items with the highest value-to-weight ratio. In emergency relief logistics, this ensures that life-saving items like medicine kits are transported first, followed by food and water as space allows. The greedy algorithm is both efficient ($O(n \log n)$) and optimal for fractional cases, making it highly effective in disaster management where quick, resource-optimized decisions are critical.